

Requirements and Specifications (Part 2/2)

Martin Kellogg

Requirements and Specifications (Part 2)

Today's agenda:

- **Reading Quiz**
- Requirements elicitation
- Formal specifications
- A special, totally-unpredictable surprise

Requirements and Specifications (Part 2)

Today's agenda:

- Reading Quiz
- **Requirements elicitation**
- Formal specifications
- A special, totally-unpredictable surprise

Requirements

Today's agenda:

- Reading Quiz
- **Requirements e**
- Formal specifica
- A special, totally

Announcements:

- Team repos created
 - accept our invitation ASAP
 - if you didn't get one, contact your TA

Requirements

Today's agenda:

- Reading Quiz
- **Requirements e**
- Formal specifica
- A special, totally

Announcements:

- Team repos created
 - accept our invitation ASAP
 - if you didn't get one, contact your TA
- Schedule your **mentor kickoff meeting**
 - first meeting must happen within one week of team notification = by end of day on Monday

Requirements

Today's agenda:

- Reading Quiz
- **Requirements**
- Formal specification
- A special, totally

Announcements:

- Team repos created
 - accept our invitation ASAP
 - if you didn't get one, contact your TA
- Schedule your **mentor kickoff meeting**
 - first meeting must happen within one week of team notification = by end of day on Monday
- **Deployment demo** during second meeting w/ TA mentor

Requirements

Today's agenda:

- Reading Quiz
- **Requirements**
- Formal specification
- A special, totally

Announcements:

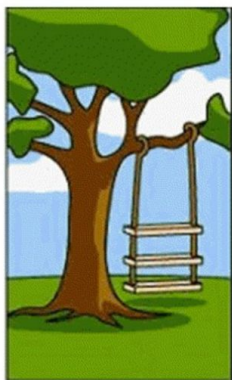
- Team repos created
 - accept our invitation ASAP
 - if you didn't get one, contact your TA
- Schedule your **mentor kickoff meeting**
 - first meeting must happen within one week of team notification = by end of day on Monday
- **Deployment demo** during second meeting w/ TA mentor
- **Preliminary plan** by October 7

Requirements

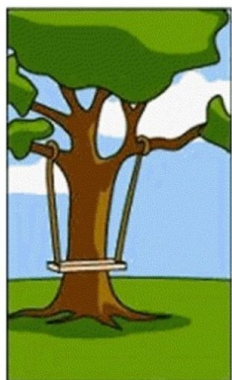
- The document I gave you during the GroupThink game was an example of a set of **requirements**

Requirements

- The document I gave you during the GroupThink game was an example of a set of **requirements**
 - **where** do requirements come from?
 - **what kinds** of requirements are there?
 - **why** is this related to specification?



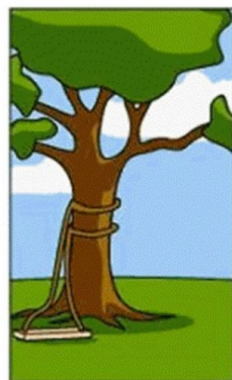
How the customer explained it



How the project leader understood it



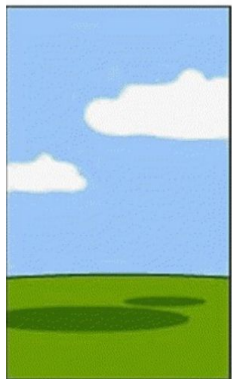
How the engineer designed it



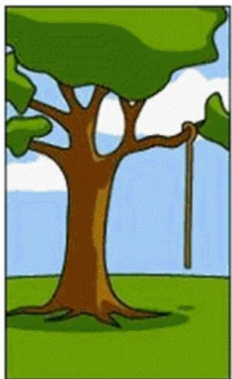
How the programmer wrote it



How the sales executive described it



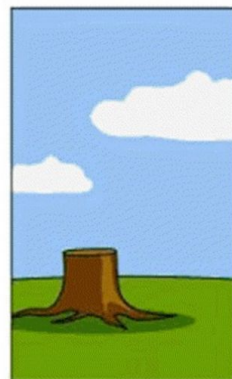
How the project was documented



What operations installed



How the customer was billed



How the help desk supported it



What the customer really needed

Requirements elicitation

- Option 1: **users tell** developers what they want
 - Client determines the problem and the solution
 - Requirements might be formally provided in the form of a contract or statement of work
 - Client might provide all requirements, or just some subset (e.g., “must be HIPAA compliant”)

Requirements elicitation

- Option 1: **users tell** developers what they want
 - Client determines the problem and the solution
 - Requirements might be formally provided in the form of a contract or statement of work
 - Client might provide all requirements, or just some subset (e.g., “must be HIPAA

Not always possible: clients often
don't know what they want

Requirements elicitation

- Option 2: **direct research**
 - Interview users, ask questions about their problems, propose potential solutions, examine those solutions
 - Embed your client in your design team, or better yet, become an **anthropologist** in your client's environment
 - Build requirements documents that demonstrate your understanding of the requirements, iterate

Requirements elicitation

- Option 2: **direct research**
 - Interview users, ask questions about their problems, propose potential solutions, examine those solutions
 - Embed your client in your design team, or better yet, become an **anthropologist** in your client's environment
 - Build requirements documents that demonstrate your understanding of the requirements, iterate
 - Empowers your team with **credibility and authority**

What kinds of requirements are you eliciting?

Definition: a *functional specification* is a description of what a system should do that doesn't specify how the system should do it

What kinds of requirements are you eliciting?

Definition: a *functional specification* is a description of what a system should do that doesn't specify how the system should do it

- Input, Output
- Interface
- Response to events
- etc.

What kinds of requirements are you eliciting?

Definition: a *functional specification* is a description of what a system should do that doesn't specify how the system should do it

- Input, Output
- Interface
- Response to events
- etc.

Properties of a good functional spec:

What kinds of requirements are you eliciting?

Definition: a *functional specification* is a description of what a system should do that doesn't specify how the system should do it

- Input, Output
- Interface
- Response to events
- etc.

Properties of a good functional spec:

- **Completeness:** All requirements are documented

What kinds of requirements are you eliciting?

Definition: a *functional specification* is a description of what a system should do that doesn't specify how the system should do it

- Input, Output
- Interface
- Response to events
- etc.

Properties of a good functional spec:

- **Completeness:** All requirements are documented
- **Consistency:** No conflicts between requirements

What kinds of requirements are you eliciting?

Definition: a *functional specification* is a description of what a system should do that doesn't specify how the system should do it

- Input, Output
- Interface
- Response to events
- etc.

Properties of a good functional spec:

- **Completeness:** All requirements are documented
- **Consistency:** No conflicts between requirements
- **Precision:** No ambiguity in requirements

What kinds of requirements are you eliciting?

- But, not all specs are functional!

What kinds of requirements are you eliciting?

- But, not all specs are functional!

Definition: A *quality requirement* specifies not the functionality of the system, but the manner in which it delivers that functionality

What kinds of requirements are you eliciting?

- But, not all specs are functional!

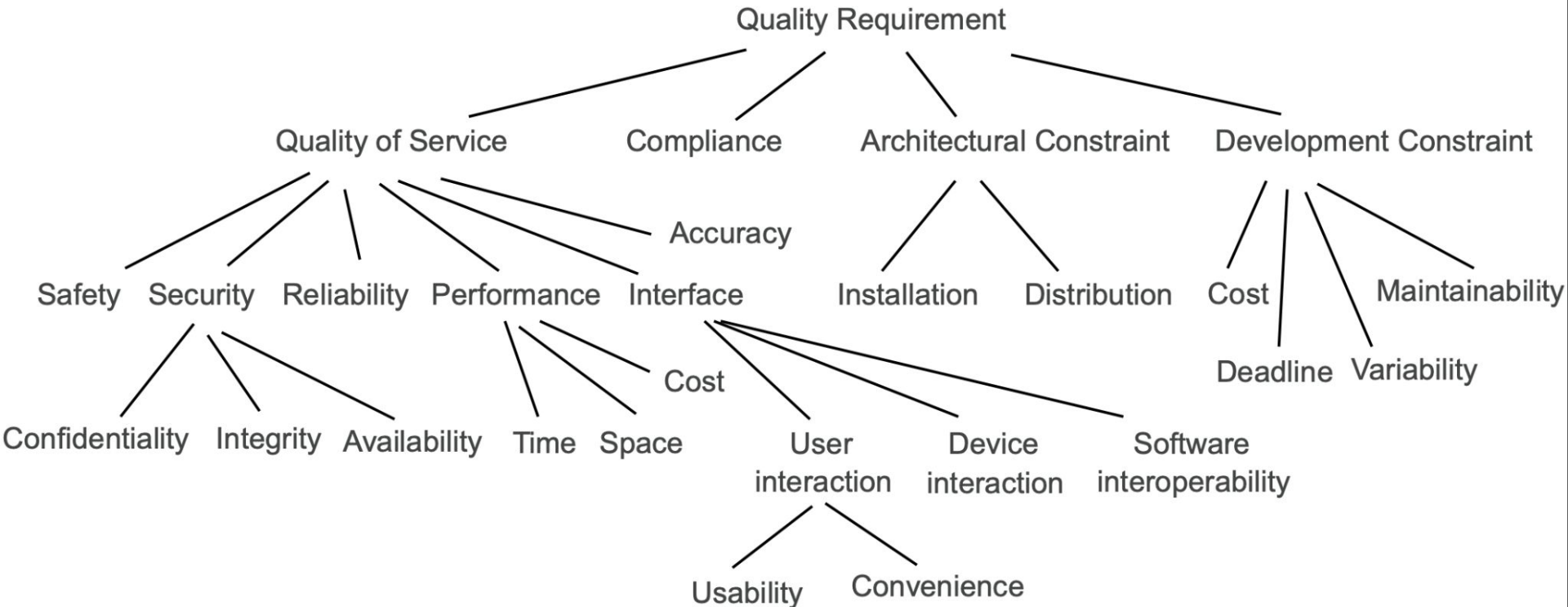
Definition: A *quality requirement* specifies not the functionality of the system, but the manner in which it delivers that functionality

Quality requirements can be more important than functional requirements:

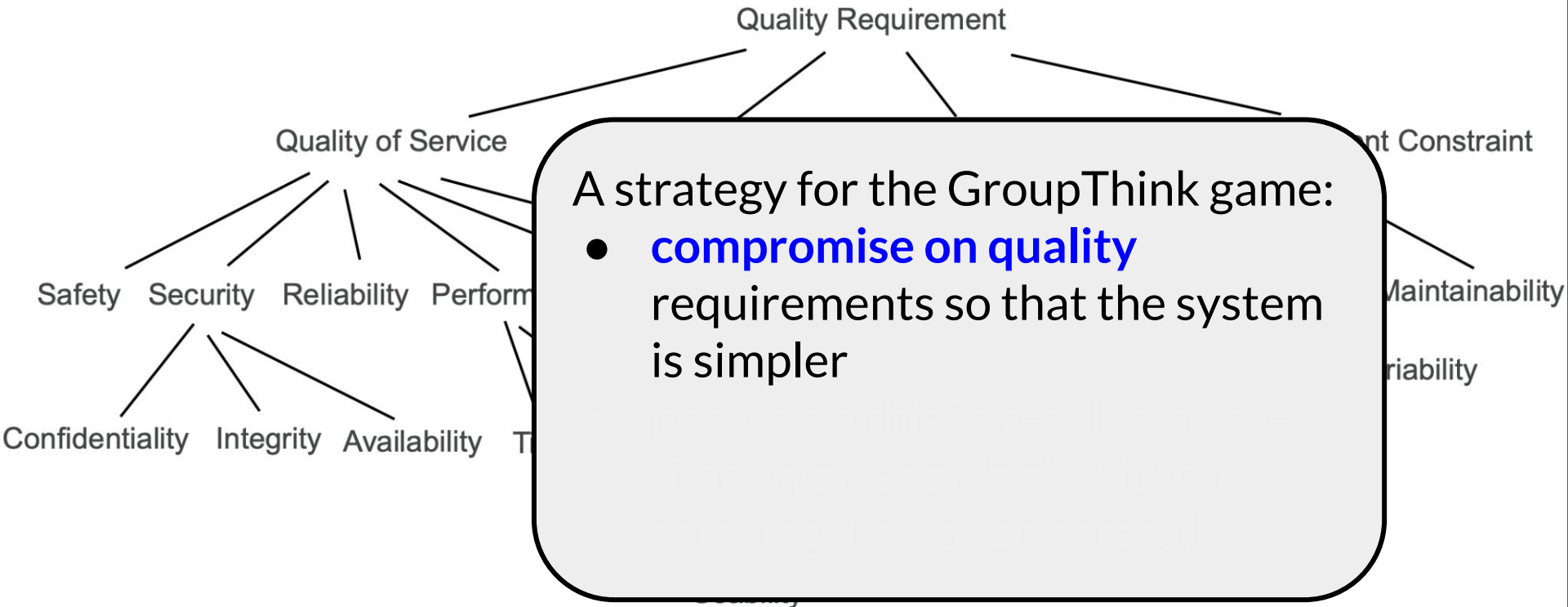
- Can work around missing functionality
- Low-quality system may be unusable

Examples of quality requirements

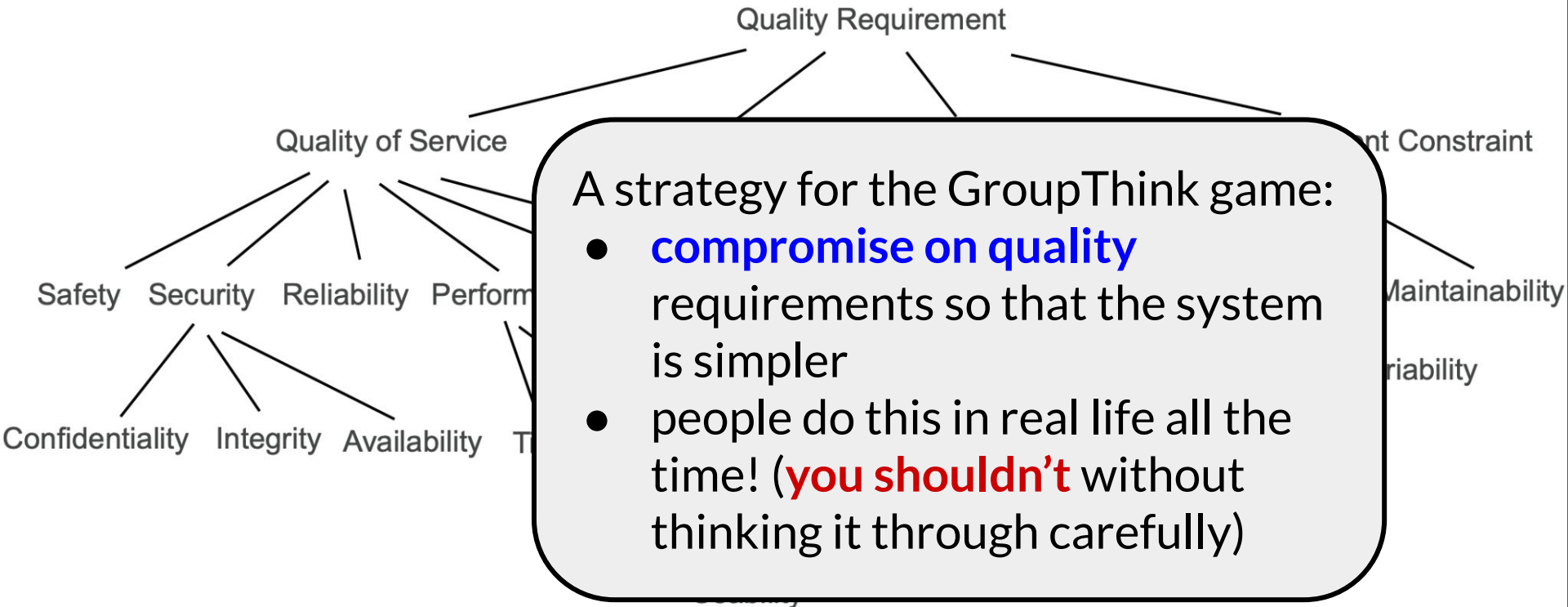
Examples of quality requirements



Examples of quality requirements



Examples of quality requirements



Can we have all the quality requirements?

- Who is going to ask for a slow, inefficient, unmaintainable system?

Can we have all the quality requirements?

- Who is going to ask for a slow, inefficient, unmaintainable system?
- A better way to think about quality requirements is as **design criteria** to help choose between alternative implementations

Can we have all the quality requirements?

- Who is going to ask for a slow, inefficient, unmaintainable system?
- A better way to think about quality requirements is as **design criteria** to help choose between alternative implementations
- The question becomes: **to what extent** must a product satisfy these requirements to be acceptable?

Can we have all the quality requirements?

- Who is going to ask for a slow, inefficient, unmaintainable system?
- A better way to think about quality requirements is as **design criteria** to help choose between alternative implementations
- The question becomes: **to what extent** must a product satisfy these requirements to be acceptable?

Trade-offs!

Expressing Quality Requirements

- Requirements serve as **contracts**: they should be **testable/falsifiable**

Expressing Quality Requirements

- Requirements serve as **contracts**: they should be **testable/falsifiable**
- An **informal** goal is a general intention (e.g., “ease of use” or “high security”)
 - May still be helpful to developers as they convey the intentions of the system users

Expressing Quality Requirements

- Requirements serve as **contracts**: they should be **testable/falsifiable**
- An **informal** goal is a general intention (e.g., “ease of use” or “high security”)
 - May still be helpful to developers as they convey the intentions of the system users
- A **verifiable** non-functional requirement is a statement using some measure that can be objectively tested

Expressing Quality Requirements

- Requirements serve as **contracts**: they should be **testable/falsifiable**
- An **informal** goal is a general (or “high security”)
 - May still be helpful to develop intentions of the system users
- A **verifiable** non-functional requirement is a statement using some measure that can be objectively tested

Advice: when possible, make your quality requirements verifiable

Informal vs. Verifiable Example

- **Informal** goal: “the system should be easy to use by experienced controllers, and should be organized such that user errors are minimized.”

Informal vs. Verifiable Example

- **Informal** goal: “the system should be easy to use by experienced controllers, and should be organized such that user errors are minimized.”
- **Verifiable** non-functional requirement: “Experienced controllers shall be able to use all the system functions after a total of **two** hours training. After this training, the average number of errors made by experienced users shall not exceed **two** per day, on average.”

Requirements and Specifications (Part 2)

Today's agenda:

- Reading Quiz
- Requirements elicitation
- **Formal specifications**
- A special, totally-unpredictable surprise

How to write specifications

Many ways:

How to write specifications

Many ways:

- as English prose (e.g., the group think game's spec)
- as semi-structured English prose (e.g., user stories)
- as structured English (*formal specification*)
- as an executable formal specification

How to write specifications

Many ways:

- as **English prose** (e.g., the group think game's spec)
- as semi-structured English prose (e.g., the group think game's spec)
- as structured English (*formal spec*)
- as an executable formal specification

Technical writing courses
should cover how to do this.
However...

Aside: Technical Writing

- Writing **clearly and concisely** is one of the most valuable skills you can have as an engineer:
 - Design docs, emails, client communications, specs, prompts...

Aside: Technical Writing

- Writing **clearly and concisely** is one of the most valuable skills you can have as an engineer:
 - Design docs, emails, client communications, specs, prompts...
- Technical communication should be **precise and to the point**
 - Unlike, e.g., your high school English class
 - Avoid flowery language, elegant variation, etc.
 - Length maximums rather than minimums are typical

Aside: Technical Writing

- Writing **clearly and concisely** is one of the most valuable skills you can have as an engineer:
 - Design docs, emails, client communications, specs, prompts...
- Technical communication should be **precise and to the point**
 - Unlike, e.g., your high school English class
 - Avoid flowery language, elegant variation, etc.
 - Length maximums rather than minimums are typical
- One trick I find useful: **length-to-content ratio**
 - If you can say something in fewer syllables, then do

Aside: Technical Writing: LCR

- In general, if a sentence means the same thing without a word, cut that word

Aside: Technical Writing: LCR

- In general, if a sentence means the same thing without a word, cut that word
- Always avoid intensifier adverbs like “very”, “extremely”, etc.

Aside: Technical Writing: LCR

- In general, if a sentence means the same thing without a word, cut that word
- Always avoid intensifier adverbs like “very”, “extremely”, etc.
- Avoid complex tenses: stick to present or simple perfect

Aside: Technical Writing: LCR

- In general, if a sentence means the same thing without a word, cut that word
- Always avoid intensifier adverbs like “very”, “extremely”, etc.
- Avoid complex tenses: stick to present or simple perfect
- Substitute common words or phrases with shorter words/phrases with the same meaning, e.g.:
 - utilize -> use, is/are able to -> can, might be able to -> could, could possibly -> might, demonstrate/illustrate -> show, the following -> below, consequently -> thus/so, etc.

Aside: Technical Writing: LCR

- In general, if a sentence means the same thing without a word, cut that word.

- Always use the active voice.

- Avoid contractions.

- Substitute numbers for words.
- Substitute words for numbers.

- utilize → use, could → might, demonstrate/illustrate → show, the following → below, consequently → thus/so, etc.

In-class exercise: apply LCR to this sentence:

“We demonstrate in the following that developers are able to utilize our technique to improve their productivity by a factor of 1.7x.”

Aside: Technical Writing: LCR

- In general, if a sentence means the same thing without a word, cut that word.

- Always use the active voice.

- Avoid complex sentences.

- Substitute "show" for "demonstrate".

- Substitute "show" for "demonstrate".
 - utilize → use

could possibly → might, demonstrate/illustrate → show, the following → below, consequently → thus/so, etc.

How to write specifications

Many ways:

- as English prose (e.g., the group think game's spec)
- as semi-structured English prose (e.g., **user stories**)
- as structured English (*formal specification*)
- as an executable formal specification

Review: user stories

“As a <role>, I can <capability>, so that I can <receive benefit>.”

Review: user stories

“As a <role>, I can <capability>, so that I can <receive benefit>.”

- also requires a *condition of satisfaction*, which is the measurement you will use to decide if the user story has been completed

Review: user stories

“As a <role>, I can <capability>, so that I can <receive benefit>.”

- also requires a *condition of satisfaction*, which is the measurement you will use to decide if the user story has been completed
- examples:
 - “As a **typical computer user**, I want to **specify folders to backup**, so that **my most important files are safe**”
 - “As a **power user**, I want to **specify subfolders and filetypes NOT to backup**, so that **my backup doesn't fill up with things that I don't need to preserve**”

How to write specifications

Many ways:

- as English prose (e.g., the group think game's spec)
- as semi-structured English prose (e.g., user stories)
- as **structured English** (*formal specification*)
- as an executable formal specification

Formal specifications

Definition: a *formal specification* is a model of a system's design expressed in a formal language

Formal specifications

Definition: a *formal specification* is a model of a system's design expressed in a formal language

- today's reading mostly was focused on formal specifications
- formal specifications are common in some safety-critical domains (e.g., aerospace, automotive software)
- to build one, you typically need to invest in learning a formal specification language (e.g., [TLA+](#), which is the only one I've seen used in industry)

Formal specifications

Definition: a *formal specification* is a model of a system's design expressed in a formal language

- today's reading mostly was
- formal specifications are common (e.g., aerospace, automotive)
- to build one, you typically need a specification language (e.g., used in industry)

This class doesn't cover formal specifications in any detail, but you should be aware of their existence: writing a model of your system (in any specification language, or none at all) is a good way to catch **design errors**.

How to write specifications

Many ways:

- as English prose (e.g., the group think game's spec)
- as semi-structured English prose (e.g., user stories)
- as structured English (*formal specification*)
- as an **executable formal specification**

Executable formal specifications

It is sometimes possible to *refine* a formal specification into a program.

- such specifications are usually written in a special-purpose programming language (“*interactive proof assistant*”)
- allows you to write proofs that directly apply to your executable code
- much, much more labor-intensive to develop than a standard software project
- area of active research!

Requirements and Specifications (Part 2)

Today's agenda:

- Reading Quiz
- Requirements elicitation
- Formal specifications
- **A special, totally-unpredictable surprise**

Requirements and Specifications (Part 2)

Today's agenda:

- Reading Quiz
- Requirements elicitation
- Formal specifications
- ~~A special, totally unpredictable surprise~~ GroupThink
Specification Exercise, round 2

Re-form your groups from Monday

You have (time left in class - 15) minutes to discuss the spec again

After that, we'll play another round of the game, with a new rule:

- Answers that contradict the specification count for zero, even if you all answer together
- You'll only have 30 seconds to answer each question

If you were not in class on Monday, find your group project teammates.

Takeaways: requirements and specifications

- Make sure you build the right thing (spend time gathering requirements)
- Specifications can help to:
 - increase understanding of system requirements between engineers and customer
 - document what the system does/will do
 - improve code quality
- Writing good specifications and getting everyone to understand them is hard and therefore worth spending time on